

Homework 1

Being Familiar with Python

Project Overview

Build a **Student Grade Management System** using procedural programming to showcase your grasp of core Python principles. This console-based app should handle student records, course details, and grade computations. **Optionally**, you can also incorporate classes, develop the project in a Python virtual environment instead of the base one, and share your source code on your personal GitHub repository.

Learning Objectives

This project evaluates students' understanding of:

- Variables and data types
- Strings and string manipulation
- Console input/output operations
- Conditional statements
- Loops
- Lists and list operations
- Dictionaries and dictionary operations
- Functions with parameters and return values
- File handling (reading/writing CSV and text files)
- Error handling with try-except blocks

Students who complete the optional components may also gain familiarity with object-oriented programming, Python virtual environments, and Git.

Expected Learning Outcomes

Upon completing this project, students will demonstrate:

- Proficiency in using fundamental Python data structures
- Ability to design and implement modular functions
- Skills in handling file I/O operations
- Competence in creating user-friendly console applications
- Understanding of data validation and error handling
- Capability to solve real-world problems using procedural programming

Project Requirements

Core Features

1. Student Management

- Add new students
- View all students
- Search for specific students
- Update student information

2. Course Management

- Add new courses
- View all courses
- Enroll student in course

3. Grade Management

- Record grades for students in courses
- Calculate student GPAs
- Generate grade reports

4. Data Persistence

- Save data to files
- Load data from files
- Export reports to CSV

Technical Requirements

- Use **dictionaries** to store students and courses
- Use **lists** for storing multiple grades and collections
- Implement **functions** for each major operation
- Use **file handling** for data storage
- Include **input validation** and **error handling**

Detailed Specifications

Data Structures

```
# Students will be stored in a dictionary where:
# key = student_id, value = dictionary with student details
students = {
    "S001": {
        "name": "John Doe",
        "email": "john@university.edu",
        "grades": {
            "CS101": [85, 90, 88],
            "MATH201": [92, 87]
        }
    }
}

# Courses will be stored in a dictionary where:
# key = course_code, value = dictionary with course details
courses = {
    "CS101": {
```

```

        "name": "Introduction to Programming",
        "credits": 3
    }
}

```

Required Functions

Students must implement these functions:

1. Student Management Functions

- `add_student(students, student_id, name, email)`
- `display_all_students(students)`
- `search_student(students, student_id)`
- `update_student(students, student_id, new_name, new_email)`

2. Course Management Functions

- `add_course(courses, course_code, course_name, credits)`
- `display_all_courses(courses)`
- `get_course_credits(courses, course_code)`

3. Grade Management Functions

- `enroll_student(students, student_id, course_code)`
- `record_grade(students, student_id, course_code, grade)`
- `calculate_course_average(students, course_code)`
- `calculate_student_gpa(students, courses, student_id)`

4. File Operations Functions

- `save_data_to_file(students, courses, filename)`
- `load_data_from_file(filename)`
- `export_grades_to_csv(students, courses, filename)`

5. Utility Functions

- `validate_grade(grade)` (Grade should be a float number between 0 to 100)
- `validate_email(email)` (Email should contains the at sign [@] symbol following by a mail-server name that contains the dot [.] character)
- `display_menu()`

Project Implementation Guide

Main Program Structure

```

# Initialize data structures
students = {}
courses = {}

# Load existing data
load_data_from_file(students, courses, "data.txt")

while True:

```

```

display_menu()
choice = input("Enter your choice: ")

if choice == "1":
    # Add student
    student_id = input("Enter student ID: ")
    name = input("Enter student name: ")
    email = input("Enter student email: ")
    add_student(students, student_id, name, email)

    # ... other menu options

elif choice == "5":
    # Add course
    course_code = input("Enter course code: ")
    course_name = input("Enter course name: ")
    credits = int(input("Enter credits: "))
    add_course(courses, course_code, course_name, credits)

    # ... other menu options

elif choice == "0":
    # Save and exit
    save_data_to_file(students, courses, "data.txt")
    break

```

Menu System

```

=== STUDENT GRADE MANAGEMENT SYSTEM ===
1. Add New Student
2. View All Students
3. Search Student
4. Update Student Information
5. Add New Course
6. View All Courses
7. Enroll Student in Course
8. Record Grade
9. Generate Student Report
10. Generate Course Report
11. Save Data to File
12. Load Data from File
13. Export to CSV
0. Exit

```

Sample Operations

Add New Student

```
=====
STUDENT GRADE MANAGEMENT SYSTEM
=====
1. Add New Student
2. View All Students
3. Search Student
4. Update Student Information
5. Add New Course
6. View All Courses
7. Enroll Student in Course
8. Record Grade
9. Generate Student Report
10. Generate Course Report
11. Save Data to File
12. Load Data from File
13. Export to CSV
0. Exit
-----
Enter your choice (0-13): 1
Enter student ID: 4043101
Enter student name: Hossein Homaei
Enter student email: homaei@modares.ac.ir
Student Hossein Homaei added successfully!
```

Add New Course

```
-----
Enter your choice (0-13): 5
Enter course code: AML101
Enter course name: Applied Machine Learning
Enter credits: 3
Course Applied Machine Learning added successfully!
```

View All Courses

ALL COURSES		
Course Code	Course Name	Credits
AML101	Applied Machine Learning	3
SS5105	Secure Software Systems	3

Enroll Student in Course

```
-----
Enter your choice (0-13): 7
Enter student ID: 4043101
Enter course code: SS5105
Student Hossein Homaei enrolled in SS5105 successfully!
```

View All Students

ALL STUDENTS			
Student ID	Name	Email	Courses
4043101	Hossein Homaei	homaei@modares.ac.ir	2
4023103	Ali Fimailli	a_fimailli@modares.ac.ir	1
4023104	Sina Lakzari	sinalakzare@gmail.com	1

Record Grade

```
-----
Enter your choice (0-13): 8
Enter student ID: 4043101
Enter course code: AML101
Enter grade: 100
Grade 100.0 recorded for Hossein Homaei in AML101!
```

Search Student

```
-----
Enter your choice (0-13): 3
Enter student ID to search: 4043101
-----
STUDENT DETAILS - Hossein Homaei (4043101)
-----
Name: Hossein Homaei
Email: homaei@modares.ac.ir
Enrolled in 2 course(s)
-----
Course Grades:
-----
Course      Grades      Average      Letter
-----
AML101      100.0, 90.0      95.00      A
SS5105      100.0              100.00      A
-----
Overall GPA: 4.00
```

Update Student Information

```
-----
Enter your choice (0-13): 4
Enter student ID to update: 4023104
Enter new name (leave blank to keep current):
Enter new email (leave blank to keep current): sinalakzare@gmail.com
Student information updated successfully!
```

Generate Student Report

```
-----
Enter your choice (0-13): 9
Enter student ID: 4043101
-----
STUDENT REPORT - Hossein Homaei
-----
Student ID: 4043101
Email: homaei@modares.ac.ir
Overall GPA: 4.00
-----
Course      Course Name      Average      Grade      Credits
-----
AML101      Applied Machine Learning      95.00      A      3
SS5105      Secure Software Systems      100.00      A      3
-----
```

Generate Course Report

```

=====
Enter your choice (0-13): 10
Enter course code: AML101
=====
COURSE REPORT - Applied Machine Learning (AML101)
=====
Course Name: Applied Machine Learning
Credits: 1
Enrolled Students: 1

Statistics:
Average Grade: 95.00
Highest Grade: 95.00
Lowest Grade: 95.00
Students with Grades: 1

Grade Distribution:
A: 1 students (100.0%)
B: 0 students (0.0%)
C: 0 students (0.0%)
D: 0 students (0.0%)
F: 0 students (0.0%)
=====

```

Save Data to File

```

=====
Enter your choice (0-13): 11
Enter filename (default: grade_data.txt):
Data saved successfully to grade_data.txt!
=====

```

Load Data from File

```

=====
Enter your choice (0-13): 12
Enter filename (default: grade_data.txt):
Data loaded successfully from grade_data.txt!
=====

```

Export to CSV result file

G	F	E	D	C	B	A	h
Letter Grade	Average	Grades	Course Name	Course Code	Student Name	Student ID	1
A	95	100.0/90.0	Applied Machine Learning	AML101	Hossein Homaei	4043101	2
A	100	100	Secure Software Systems	SSS105	Hossein Homaei	4043101	3
							4

Sample Function Implementations

Example 1: Adding a Student

```

def add_student(students, student_id, name, email):
    """Add a new student to the system"""
    if student_id in students:
        print("Error: Student ID already exists!")
        return False

    if not validate_email(email):
        print("Error: Invalid email format!")
        return False

    students[student_id] = {
        "name": name,
        "email": email,
        "grades": {} # Empty dictionary for grades
    }
    print(f"Student {name} added successfully!")
    return True

```

Example 2: Calculating GPA

```

def calculate_student_gpa(students, courses, student_id):
    """Calculate GPA for a student"""
    if student_id not in students:
        return None

    total_points = 0
    total_credits = 0

```

```

    for course_code, grades in students[student_id]["grades"].items():
        if course_code in courses and grades: # Check if course exists
            and has grades
                course_avg = sum(grades) / len(grades)
                letter_grade = convert_to_letter_grade(course_avg) # A if
                >= 90, B else if >= 80, C else if >= 70, D else if >= 60, F otherwise
                grade_points = convert_to_grade_points(letter_grade) # A =
                4, B = 3, C = 2, D = 1, F = 0
                credits = courses[course_code]["credits"]

                total_points += grade_points * credits
                total_credits += credits

    return total_points / total_credits if total_credits > 0 else 0.0

```

Example 3: File Operations

```

def save_data_to_file(students, courses, filename):
    """Save students and courses data to a file"""
    try:
        with open(filename, 'w') as file:
            # Save courses first
            file.write("COURSES\n")
            for course_code, course_info in courses.items():
                file.write(f"{course_code},{course_info['name']},
{course_info['credits']}\n")

            # Save students and their grades
            file.write("STUDENTS\n")
            for student_id, student_info in students.items():
                file.write(f"{student_id},{student_info['name']},
{student_info['email']}\n")

            # Save grades for this student
            for course_code, grades in
student_info['grades'].items():
                grades_str = ','.join(map(str, grades))
                file.write(f"GRADES,{student_id},{course_code},
{grades_str}\n")

        print("Data saved successfully!")
    except Exception as e:
        print(f"Error saving data: {e}")

```

Deliverables

Students should submit:

1. **Complete Python script** (`grade_manager.py`)
2. **Sample data files** demonstrating functionality
3. **Documentation** explaining how to use the system
4. **Error handling examples** showing robust operation

